

05 — CONCURRENCY & THE C++ MEMORY MODEL

The book barely covers this; interviews at SDE-2+ lean on it hard. Know the primitives, the ownership of data, and enough of the memory model to reason about correctness.

How to think about this. Stop imagining threads as sequential programs running side by side. Instead hold two facts: (1) the compiler and CPU reorder your memory operations for speed, and (2) each core sees other cores' writes on a delay. "Correct" concurrent code is code where you've established enough *happens-before* edges (via mutex, or acquire/release) that the reorderings you allowed can't produce a state your logic didn't expect. Every primitive in this file is just a way to buy a happens-before edge.

Take-home lesson. Don't reason about "threads"; reason about *shared mutable state*. Concurrency bugs live exactly where two threads touch the same data and at least one writes. Eliminate the sharing (copy, or confine to one thread), make it immutable, or serialize access — those are the only three tools, and they're in that preference order.

1. THREADS & THE BASIC TOOLKIT

- `std::thread` — you must `join()` or `detach()` before it's destroyed, else `std::terminate`. Prefer `std::jthread` (C++20): auto-joins in dtor and carries a `stop_token` for cooperative cancellation.

```
// C++20 jthread: cooperative cancellation + automatic join (RAII for threads):
std::jthread worker([](std::stop_token st) {
    while (!st.stop_requested()) { /* do a unit of work */ }
});
// ...no explicit join needed; worker's destructor requests stop AND joins.
```

- `std::async` + `std::future` — launch a task, get the result later.
`std::launch::async` forces a new thread; `deferred` runs on `get()`.
 Gotcha: the future returned by `std::async` blocks in its destructor until the task finishes (a common surprise).
 - `std::promise` / `std::future` — one-shot channel for a value or exception across threads. `std::packaged_task` wraps a callable + future.
-

2. DATA RACES VS RACE CONDITIONS

- Data race = two threads access the same memory, at least one writes, with no synchronization and not both atomic → undefined behavior. Not "maybe wrong" — UB.
 - Race condition = a logic bug where correctness depends on timing (can exist even in race-free code).
 - The fix for data races: mutual exclusion (mutex) or atomics.
-

3. MUTEXES & LOCKS

- `std::mutex`, `recursive_mutex`, `timed_mutex`, `shared_mutex` (C++17, reader/writer: many `shared_lock` readers OR one `unique_lock` writer).
- Always lock via RAII wrappers, never manual lock/unlock:
 - `std::lock_guard` — simplest, scope-bound.
 - `std::unique_lock` — movable, deferrable, works with condition variables, supports manual unlock/relock. Slightly heavier.

- `std::scoped_lock` (C++17) — locks multiple mutexes deadlock-free.
- Deadlock avoidance: always acquire locks in a consistent global order, or use `std::scoped_lock` / `std::lock` to grab several atomically. Classic four Coffman conditions: mutual exclusion, hold-and-wait, no preemption, circular wait — break any one.

```
std::scoped_lock lk(m1, m2);    // deadlock-free multi-lock
```

4. CONDITION VARIABLES

Wait for a condition without busy-spinning. Always use the predicate overload to handle spurious wakeups.

```
std::unique_lock lk(m);  
cv.wait(lk, []{ return ready; });    // re-checks predicate; immune to  
spurious wakeup  
// ... producer: { lock; ready=true; } cv.notify_one();
```

- `notify_one` vs `notify_all`. Modify the shared state under the lock before notifying, or you get lost wakeups.

Stop and think. Why does `cv.wait(lk)` *without* a predicate risk waking up to do nothing — or worse, missing a notification entirely?

► answer

5. ATOMICS & `STD::ATOMIC`

- `std::atomic<T>` — indivisible ops, no torn reads/writes; `is_lock_free()` tells you if it's truly lock-free (usually for pointer/integral sizes).

Atomicity is per-operation, not per-statement. `atomic` guarantees each individual load and store is indivisible (no torn value), but `counter = counter + 1;` is *three* operations — load, add, store — with a gap where another thread can interleave, so increments get lost. For a correct

increment you need one indivisible read-modify-write: `counter.fetch_add(1)` (or `++counter`, which maps to it). CAS (`compare_exchange`) is the general tool when the update isn't a simple arithmetic RMW.

- `fetch_add`, `compare_exchange_weak/strong` (CAS — the foundation of lock-free algorithms). `weak` can fail spuriously (loop it); `strong` doesn't.
- `std::atomic_flag` — the only guaranteed-lock-free type; basis of spinlocks.
- `shared_ptr` control-block ref counts are atomic, but the pointee and the `shared_ptr` object itself are not thread-safe — use `atomic<shared_ptr>` (C++20) or external sync for the handle.

6. THE MEMORY MODEL & ORDERING (THE HARD PART)

The compiler and CPU reorder memory ops. Ordering args to atomic ops constrain what other threads can observe.

Order	Meaning	Use
<code>relaxed</code>	atomicity only, no ordering	counters where order doesn't matter
<code>acquire</code>	no reads/writes <i>after</i> can move before (on loads)	lock acquire, consume-side
<code>release</code>	no reads/writes <i>before</i> can move after (on stores)	lock release, publish-side
<code>acq_rel</code>	both (on RMW)	CAS in a lock
<code>seq_cst</code>	single total order across all <code>seq_cst</code> ops (default)	when in doubt

- acquire/release pairing establishes a *happens-before*: everything before a release store in thread A is visible to thread B after it acquire-loads the same atomic. This is how you publish data cheaply without a full fence.
- `seq_cst` is the default and safest; it's also the slowest. Say "I'd start with `seq_cst` and relax only with a benchmark + a proof."

```
// acquire/release publish pattern — the workhorse of lock-free
publishing:
std::atomic<Data*> ptr{nullptr};
// producer:
Data* d = new Data(...); // (1) build fully
ptr.store(d, std::memory_order_release); // (2) publish — nothing above
(1) reorders past here
// consumer:
if (Data* p = ptr.load(std::memory_order_acquire)) // pairs with the
release
    use(*p); // guaranteed to see the fully-
built Data
```

Stop and think. Why is `relaxed` enough for a shared counter but *not* for the pointer publish above?

► answer

- `volatile` is NOT for threading — it prevents *compiler* optimizations for memory-mapped I/O; it gives no atomicity and no cross-thread ordering. Naming this correctly is a common interview filter.

Take-home lesson. `volatile` means "this memory can change behind the compiler's back" (hardware registers, signal handlers). `atomic` means "this access is indivisible and ordered across threads." They solve different problems; using `volatile` for thread communication is a classic C-holdover bug that gives you neither atomicity nor a memory fence.

7. PRACTICAL PATTERNS & HAZARDS

- `std::call_once` / `std::once_flag` — thread-safe one-time init (or just use a magic static).
- False sharing — two threads hammering different variables that share a cache line serialize on the coherency protocol. Fix: pad / align to `std::hardware_destructive_interference_size`.
- Thread pool — reuse N threads over a task queue (CV + mutex + queue). Know how to sketch one.

- Lock-free is hard: ABA problem (pointer reused, CAS wrongly succeeds), memory reclamation (hazard pointers / RCU). Mention these to show awareness; don't claim lock-free by default — it's often slower and buggier than a well-scoped mutex.
-

SELF-CHECK LADDER (GRADE YOUR OWN MASTERY)

Level 1 — Recall

- Define data race vs race condition in one sentence each.
- Name the memory orders from weakest to strongest.
- What does `std::jthread` do that `std::thread` doesn't?

Level 2 — Apply

1. Implement a thread-safe queue (mutex + condition_variable, predicate wait).
2. Explain acquire/release with a producer publishing a pointer to data (write the code).
3. Sketch a fixed-size thread pool; then add graceful shutdown via `stop_token`.

Level 3 — Transfer

4. Why must you modify shared state *under the lock* before `notify_one`? Tie it to lost wakeups.
5. Why is `volatile bool done` insufficient for stopping a worker thread? Say precisely what guarantee it lacks.
6. Show an ABA scenario and how a tagged pointer / hazard pointer addresses it.

Level 4 — Teach

- Explain *happens-before* to someone who thinks threads just run line-by-line. If you can't produce a two-thread example where reordering breaks naive code, you're reciting terms.

CONNECTS TO

- File 01 — `shared_ptr`'s atomic ref count; who owns data across threads is a lifetime question.
- File 02 — moving work/data into a thread or task (init-capture, move-only lambdas) to avoid sharing.
- File 07 — `noexcept` and RAI lock guards; never manual lock/unlock, always scope-bound.
- File 08 — `jthread`, `atomic_ref`, `atomic<shared_ptr>`, `stop_token` are the C++20 additions used above.